Praktikum 7 – Exception Handling

A. Identitas dan Tujuan Praktikum

Praktikum hari ini kita mengubah toko online UMKM dari program yang **mati atau diam saja** saat ada masalah menjadi program yang **menjelaskan kegagalannya**. Sebelumnya, pembayaran gagal hanya dijawab `false`. Setelah praktikum ini, toko menjawab "Saldo tidak cukup: perlu Rp215000, tersedia Rp200000 (kurang Rp15000)", lalu mengembalikan stok yang sempat dipesan. Di akhir sesi praktikum kumpulkan **kode program** dan **Lembar Kerja (LK) Praktikum** yang mahasiswa isi sambil bekerja.

Mata kuliah	Pemrograman Berorientasi Objek
Topik	Exception handling (penanganan exception): try-catch-finally, checked/unchecked, custom exception, try-with-resources
Tools	Java 25 (JDK 25), VS Code + Extension Pack for Java
Bentuk kerja	Individu, 120 menit
Dikumpulkan di akhir sesi	Zip folder `src` + Lembar Kerja `LK-P07-<NIM>.pdf`
Bahan	Paket `Praktikum-07-Exception.zip`
Dosen	Afdhal Dinilhak

Tujuan Praktikum

a. Menjelaskan hierarki exception Java serta perbedaan checked dan unchecked exception
b. Mengimplementasikan penanganan exception dengan try-catch-finally, multi-catch, throw/throws, dan try-with-resources
c. Merancang custom exception yang tepat untuk aturan bisnis aplikasi
d. Mengevaluasi kode penanganan exception dan memperbaiki anti-pattern

Persiapan Praktikum

Unduh dan ekstrak `Praktikum-07-Exception.zip` Isinya:

```
Praktikum-07-Exception /
├── p07-starter/          <- the project you open in VS Code (L0)
├── bahan/
│   ├── L3/ScenarioRunner.java
│   ├── L4/CatalogReader.java
│   └── L5/FragileShop.java
└── LK-P07.docx
```

Cara menjalankan program

Cara 1 – Tombol Run di VS Code. Buka file yang punya `main`, lalu klik **Run** di atas `main`. VS Code otomatis mengompilasi seluruh proyek ke folder **`bin`**.

Cara 2 – Terminal VS Code (wajib untuk program yang butuh argumen). Buka **Terminal → New Terminal**. Terminal ini sudah berada di folder proyek:

```
java -cp bin selfcheck.SelfCheck
java -cp bin shop.app.ShopApp
java -cp bin shop.demo.CrashDemo 2
java -cp bin shop.demo.CariProdukDemo abc
java -cp bin tools.NimParams
```

Cara 3 – Kompilasi manual tanpa VS Code. Hasilnya disimpan di folder `out`, jadi perintah `java` memakai `-cp out`:

- Windows (CMD): `dir /s /b src\*.java > sources.txt`, lalu `javac -d out @sources.txt`, lalu `java -cp out selfcheck.SelfCheck`.
- Linux/macOS: `javac -d out $(find src -name "*.java")`, lalu `java -cp out selfcheck.SelfCheck`.

Jika muncul `Error: Could not find or load main class ...`

Penyebab	Cara mengecek	Perbaikan
`-cp` menunjuk folder yang salah, misalnya `out` padahal VS Code memakai `bin`	Lihat isi folder proyek: ada `bin` atau `out`?	Pakai `-cp bin`
Terminal tidak berada di folder proyek	Ketik `dir` (Windows) atau `ls`; harus terlihat `src`, `bin`, dan `katalog.csv`	`cd` ke folder `exception-<NIM>`
Ada file lain yang gagal dikompilasi	Tanda merah di panel **Problems**	Perbaiki file yang disebut
Nama class salah ketik	Harus lengkap dengan package, huruf besar/kecil sama	Salin persis dari modul

Cara membaca hasil SelfCheck

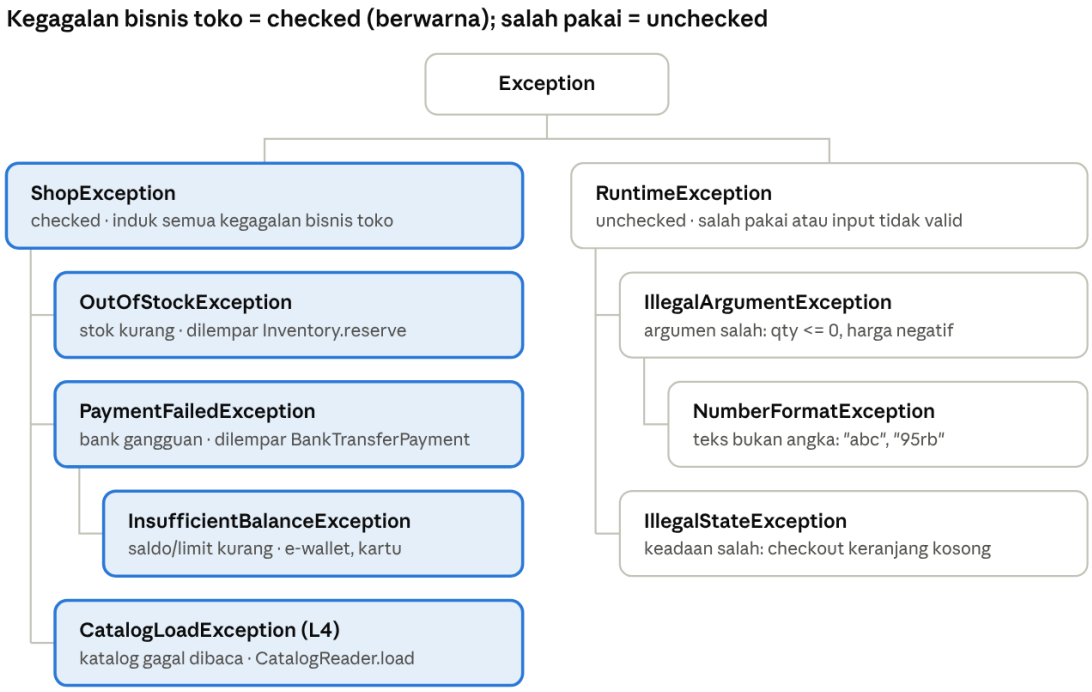
`SelfCheck` memeriksa kode mahasiswa secara otomatis. Jalankan kapan saja, butir yang belum dikerjakan memang berstatus `[GAGAL]`. Alasan kegagalan ditulis setelah tanda `->`:

Alasan	Arti	Yang perlu dilakukan
`class/interface belum ada: ...`	File belum dibuat, atau nama/package berbeda	Buat file dengan nama dan package yang persis sama
`method/konstruktor tidak ditemukan: ...`	Nama, tipe parameter, atau `public` tidak sesuai	Cocokkan tanda tangan method dengan modul
`hasil tidak sesuai spesifikasi: ...`	Kode berjalan, tetapi hasilnya salah	Baca ulang spesifikasi butir tersebut
`seharusnya X, ternyata Y`	Exception yang dilempar salah jenis, atau tidak ada exception	Periksa `throw` dan `throws` Anda
`kode Anda melempar ...` / `ShopApp crash: ...`	Kode Anda mati karena exception yang tidak ditangkap	Baca nama exception-nya; itulah petunjuknya

Jalankan SelfCheck dari folder proyek. Beberapa pemeriksaan membaca `katalog.csv` dan file sumber Anda, jadi folder kerjanya harus folder proyek (tombol Run dan terminal VS Code sudah otomatis begitu). SelfCheck memberi 50 dari 100 poin. 50 poin sisanya dari Lembar Kerja, yang menilai pemahaman mahasiswa, bukan hanya kode yang berjalan.

B. Peta Proyek Praktikum

Mahasiswa melanjutkan proyek toko dari modul Abstraction. Dua package baru muncul (`shop.exception` dan `shop.io`), dan empat file lama berubah: `pay` serta `checkout` berhenti menjawab `true`/`false` dan mulai melempar exception.



keluarga exception took: checked (berwarna) dan unchecked

Cara membaca peta ini: semua custom exception toko (kotak berwarna) diturunkan dari `ShopException`, sehingga compiler **memaksa** pemanggil menanganinya. Kesalahan pemakaian (kotak putih) tetap memakai exception bawaan Java yang unchecked. Semua kegagalan toko akhirnya ditangkap di satu tempat, yaitu `ShopApp.buy(...)`; `CheckoutService` hanya membereskan stok lalu melemparnya lagi.

Package	Isi	Status hari ini
`shop.model`	`Product`, `PhysicalProduct`, `DigitalProduct`, `Address`, `Order`	Tetap (dari modul Abstraction)
`shop.shipping`	`Shippable`, `ShippingService`, `RegularShipping`, `ExpressShipping`	Tetap
`shop.discount`	`Discountable`	Tetap
`shop.payment`	`PaymentMethod`, `EWalletPayment`, `BankTransferPayment`, `CreditCardPayment`, ...	**Diubah di L3**: `pay` menjadi `void ... throws`
`shop.exception`	`ShopException` dan turunannya	**Baru di L3 dan L4**
`shop.service`	`CheckoutService`, `Inventory`	**Diubah di L3**
`shop.io`	`CatalogReader`	**Baru di L4**
`shop.app`	`ShopApp`, `ScenarioRunner`	**Diubah di L1, L3, L4**
`shop.demo`	`CrashDemo`, `FlowDemo`, `CariProdukDemo`	Disediakan (L1, L2)
`review`	`FragileShop`	Disalin di L5
`selfcheck`, `tools`	`SelfCheck`, `NimParams`	Alat bantu, jangan diubah

Beberapa file disalin dari folder `bahan` **pada langkahnya masing-masing**, bukan di awal. Alasannya: file itu memakai class yang baru mahasiswa buat di langkah tersebut, sehingga kalau disalin lebih awal, proyek tidak bisa dikompilasi.

Langkah	Salin dari bahan	Ke proyek Anda
L3.f	`L3/ScenarioRunner.java`	`src/shop/app/ScenarioRunner.java`
L4.b	`L4/CatalogReader.java`	`src/shop/io/CatalogReader.java`
L5.a	`L5/FragileShop.java`	`src/review/FragileShop.java`

C. Latihan Praktikum

Setiap langkah berisi tujuan, pengingat singkat, langkah kerja, kesalahan umum, contoh output, checkpoint SelfCheck, dan kotak **Isi LK**. Bagian bertanda **TULIS SENDIRI** wajib mahasiswa kerjakan dari spesifikasi. Kode yang diberikan lengkap boleh diketik ulang atau disalin, tetapi pastikan Anda memahaminya, karena LK menanyakannya. Semua uang memakai `long` dalam rupiah.

L0 – Persiapan (10 menit)

Tujuan: membuka proyek starter, mengisi identitas, dan memastikan program berjalan di Java 25.

Berbeda dengan modul Abstraction, hari ini kita **tidak** mulai dari folder kosong. Proyek starter berisi kode toko di akhir modul Abstraction (masih memakai `boolean` untuk pembayaran), ditambah program demo dan alat bantu.

Langkah 1 – Siapkan folder proyek. Salin folder `p07-starter` dari zip ke folder kerja Anda, misalnya `D:\praktikum-oop`, lalu ganti namanya menjadi `exception-<NIM>`, misalnya `exception-2401010057`.

Langkah 2 – Buka di VS Code. Pilih **File → Open Folder**, lalu pilih folder `exception-<NIM>` itu sendiri, bukan folder `src` di dalamnya. Struktur yang terlihat:

```
exception-2401010057/
├── .vscode/settings.json   <- source folder = src, output folder = bin
├── katalog.csv             <- used in L4 (keep it next to src, not inside)
└── src/
    ├── selfcheck/SelfCheck.java
    ├── tools/NimParams.java
    └── shop/
        ├── StudentInfo.java
        ├── app/ShopApp.java
        ├── demo/      CrashDemo, FlowDemo, CariProdukDemo
        ├── discount/  model/  shipping/      (from the Abstraction lab)
        ├── payment/   PaymentMethod (still boolean), EWallet, BankTransfer,
        │   CreditCard
        └── service/   CheckoutService (still boolean), Inventory (TODO)
```

Langkah 3 – Pastikan Java 25. Buka **Terminal → New Terminal**, ketik `java -version`. Baris pertama harus menyebut versi `25`. Jika VS Code memakai JDK lain, tekan `Ctrl+Shift+P`, pilih **Java: Configure Java Runtime**, lalu pilih JDK 25. Ini penting: di L1 Anda memakai fitur `catch (... _)` yang baru tersedia sejak Java 22.

Langkah 4 – Isi identitas. Buka `src/shop/StudentInfo.java`, ganti dua baris berikut dengan NIM (angka saja) dan nama Anda:

```
public static final String NIM = "0000000000";      // TODO L0: your NIM
public static final String NAME = "ISI NAMA ANDA";    // TODO L0: your name
```

NIM Anda menentukan **saldo awal e-wallet** dan **stok awal** produk, sehingga hasil program Anda berbeda dengan teman sebelah. Jalankan `java -cp bin tools.NimParams` untuk melihatnya. Contoh untuk NIM 2401010057:

```
Parameter praktikum P07 untuk NIM 2401010057
  Saldo awal e-wallet DompetKu : Rp200000
  Stok awal KOPI-001           : 3
  Stok awal RDG-002            : 2
  Stok awal produk lain        : 5
```

Langkah 5 – Jalankan SelfCheck pertama. Buka `src/selfcheck/SelfCheck.java`, klik **Run**. Hasil yang benar saat ini adalah `2 / 24`:

```
=== SELF CHECK: Praktikum Exception Handling (P10) ===
NIM 2401010057 | Contoh Mahasiswa | Java 25

-- L0 - Persiapan
  [LULUS] L0.1 Program berjalan di Java 25 atau lebih baru
  [LULUS] L0.2 NIM dan nama di StudentInfo sudah diisi

-- L1 - Membaca crash dan menangani input
  [GAGAL] L1.1 ShopApp tidak crash saat nomor produk berupa huruf
          -> ShopApp crash: java.lang.NumberFormatException: For input string: "abc"
...
=== HASIL: 2 / 24 pemeriksaan lulus (nilai SelfCheck 4 / 50) ===
```

Perhatikan: SelfCheck sendiri sudah menunjukkan program toko **crash**. Itulah yang diperbaiki di L1.

Kesalahan umum L0

Gejala	Penyebab	Perbaikan
`The declared package "shop.app" does not match the expected package`	Yang dibuka folder `src` atau folder induk	**File → Open Folder** ke folder `exception-<NIM>`
L0.1 gagal: "Java yang dipakai versi 21"	VS Code memakai JDK lama	**Java: Configure Java Runtime** → JDK 25
L0.2 gagal	NIM berisi huruf/spasi, atau nama belum diganti	NIM hanya angka; nama tidak boleh diawali "ISI"
Tidak ada tombol **Run** di atas `main`	Extension Pack for Java belum aktif atau belum selesai memuat proyek	Tunggu ikon Java di status bar selesai berputar, atau buka ulang folder

> ***Isi LK – bagian L0:*** *nama, NIM, kelas, versi Java dari `java -version`, dan empat parameter dari `NimParams`.*

L1 – Membaca crash dan menangani input (15 menit)

Tujuan: membaca stack trace (jejak pemanggilan) untuk menemukan sumber exception, lalu membuat `ShopApp` tidak mati saat pembeli salah mengetik.

Exception adalah laporan masalah yang dilempar saat program **berjalan**. Kalau tidak ada yang menangkapnya, program berhenti dan Java mencetak stack trace. Cara membacanya: (1) baris pertama = jenis dan pesan; (2) cari baris pertama yang diawali `shop.` = tempat masalah muncul di kode Anda; (3) baca ke bawah = siapa yang memanggil.

L1.a – Bedah tiga crash. `CrashDemo` berisi tiga skenario yang sengaja dibuat mati. Jalankan satu per satu dari terminal:

```
java -cp bin shop.demo.CrashDemo 1
java -cp bin shop.demo.CrashDemo 2
java -cp bin shop.demo.CrashDemo 3
```

Contoh bedah **skenario 1** (skenario 2 dan 3 mahasiswa bedah sendiri di LK):

```
Skenario 1
Exception in thread "main" java.lang.IndexOutOfBoundsException: Index: 4 Size: 2
	at
java.base/java.util.ImmutableCollections$AbstractImmutableList.outOfBounds(ImmutableC
ollections.java:359)
	at
java.base/java.util.ImmutableCollections$List12.get(ImmutableCollections.java:612)
	at shop.demo.CrashDemo.pickProduct(CrashDemo.java:24)
	at shop.demo.CrashDemo.main(CrashDemo.java:42)
```

Yang dicari	Jawaban skenario 1
Jenis exception	`IndexOutOfBoundsException` (unchecked)
Pesan	`Index: 4 Size: 2`: katalog berisi 2 produk, diminta indeks 4
Baris kode Anda yang pertama	`CrashDemo.java:24`, method `pickProduct`, yaitu `catalog.get(number - 1)`
Dipanggil dari	`CrashDemo.java:42`, method `main`
Apakah "Selesai." tercetak?	Tidak; program berhenti di baris yang gagal

Tips: di terminal VS Code, `Ctrl+klik` tulisan `CrashDemo.java:24` untuk melompat ke barisnya. Lewati baris `java.base/...`; itu kode bawaan Java, dan nomor barisnya bisa berbeda di komputer lain.

L1.b – Rasakan sendiri crash di `ShopApp`. Jalankan `ShopApp`, lalu ketik `abc` saat diminta nomor produk. Program mati dengan `NumberFormatException` di `ShopApp.java:28`. Jalankan sekali lagi, pilih produk `2`, lalu jumlah `-2`. Kali ini tidak ada exception. Catat apa yang terjadi untuk LK.

Ini potongan kode `ShopApp` yang bermasalah:

```
System.out.print("Nomor produk (0 = selesai): ");
int choice = Integer.parseInt(in.nextLine().trim());   // "abc" -> crash
if (choice == 0) {
    break;
}
Product product = catalog.get(choice - 1);             // "9" -> crash

System.out.print("Jumlah: ");
int quantity = Integer.parseInt(in.nextLine().trim()); // "x" -> crash, "-2" -> ???
```

L1.c – Tangani nomor produk (contoh). Ganti baris `int choice = ...` dengan:

```
int choice;
try {
    choice = Integer.parseInt(in.nextLine().trim());
} catch (NumberFormatException _) { // the exception object is not needed
    System.out.println("  Input harus berupa angka.");
    continue;                        // ask again
}
```

> ***Catatan Java modern – unnamed variable _.*** *Sejak Java 22 (JEP 456), variabel yang tidak dipakai boleh dinamai* `_`*. Di sini objek exception memang tidak diperlukan, karena pesan untuk pembeli kita tulis sendiri.* `_` ***tidak*** *membuat catch kosong menjadi benar: blok catch tetap wajib melakukan sesuatu (lihat L5).*

L1.d – TULIS SENDIRI. Lengkapi sisa loop dengan aturan berikut. Pesan harus **persis** sama, termasuk dua spasi di depan, karena diperiksa **SelfCheck**.

Kondisi	Cara menangani	Pesan
Nomor bukan 0 dan di luar `1..catalog.size()`	**Cegah dengan `if`**, jangan tangkap exception index	`  Nomor produk tidak ada.`
Jumlah bukan angka	`try-catch (NumberFormatException _)`	`  Input harus berupa angka.`
Jumlah `<= 0`	**Cegah dengan `if`**	`  Jumlah harus lebih dari 0.`

Setiap kondisi diakhiri `continue` agar pembeli ditanya lagi. Mengapa nomor produk dicegah dengan `if`, sedangkan teks angka ditangkap dengan `catch`? Itu salah satu pertanyaan LK. Petunjuk: Deitel 11.4, *When to Use Exception Handling*.

Contoh output setelah L1 (teks setelah titik dua adalah ketikan pembeli):

```
== Toko Online UMKM ==
1. [Fisik] Kopi Arabika Gayo 250g - Rp85000
2. [Fisik] Rendang Kemasan 250g - Rp120000
3. [Fisik] Teh Kayu Aro 100g - Rp35000
Nomor produk (0 = selesai): abc
  Input harus berupa angka.
Nomor produk (0 = selesai): 9
  Nomor produk tidak ada.
Nomor produk (0 = selesai): 1
Jumlah: -2
  Jumlah harus lebih dari 0.
Nomor produk (0 = selesai): 1
Jumlah: 2
  + 2 x Kopi Arabika Gayo 250g
Nomor produk (0 = selesai): 0

Keranjang: 2 barang, total Reguler Rp180000
Pembayaran berhasil? true
Sisa saldo: Rp20000
```

Coba juga beli **1 kopi + 1 rendang**. Untuk NIM contoh (saldo Rp200000), hasilnya `Pembayaran berhasil? false`. Mengapa gagal? Saldo kurang? Bank gangguan? Program tidak memberi tahu. Masalah inilah yang diselesaikan di L3.

Kesalahan umum L1

Salah	Benar	Alasan
Seluruh `while` dibungkus satu `try` besar	`try` hanya di sekitar `parseInt`	Satu input salah menghentikan seluruh loop
`catch (Exception e)`	`catch (NumberFormatException _)`	Ikut menyembunyikan bug lain, misalnya `NullPointerException`

Salah	Benar	Alasan
`catch (IndexOutOfBoundsException e)` untuk nomor produk	`if (choice < 1 || choice > catalog.size())`	Kesalahan yang bisa dicegah dengan murah, dicegah, bukan ditangkap
Lupa `continue` di dalam catch	Tambahkan `continue`	Compiler menolak: *variable choice might not have been initialized*
Catch kosong `catch (NumberFormatException _) { }`	Tampilkan pesan, lalu `continue`	Pembeli tidak tahu inputnya salah

Checkpoint L1: SelfCheck L1.1–L1.3 lulus, total **5 / 24**:

```
-- L1 - Membaca crash dan menangani input
  [LULUS] L1.1 ShopApp tidak crash saat nomor produk berupa huruf
  [LULUS] L1.2 ShopApp menolak nomor produk di luar katalog
  [LULUS] L1.3 Jumlah bukan angka atau <= 0 ditolak
...
=== HASIL: 5 / 24 pemeriksaan lulus (nilai SelfCheck 10 / 50) ===
```

> ***Isi LK – bagian L1:*** *(a) tabel bedah stack trace skenario 2 dan 3; (b) apa yang terjadi dengan jumlah `-2` sebelum diperbaiki, dan mengapa tidak muncul `NumberFormatException`; (c) mengapa nomor produk dicegah dengan `if`, sedangkan teks angka ditangkap dengan `catch`.*

L2 – Alur try-catch-finally: prediksi dulu, baru jalankan (20 menit)

Tujuan: memahami urutan eksekusi `try`, `catch`, dan `finally`, termasuk saat catch tidak cocok, saat ada `return`, dan saat ada beberapa catch.

Begitu sebuah baris di `try` melempar exception, sisa blok `try` dilewati. Java memeriksa `catch` dari atas ke bawah dan menjalankan **yang pertama cocok** saja. `finally` hampir selalu jalan. Exception yang tidak tertangkap naik ke method pemanggil.

Cara kerja langkah ini (Prediksi → Jalankan → Selidiki → Ubah).

1. **Prediksi.** Baca kodenya, lalu tulis tebakan output di LK **sebelum** menjalankan program.
2. **Jalankan.** Jalankan program, lalu tempel output sebenarnya di LK.
3. **Selidiki.** Untuk setiap baris yang berbeda, jelaskan penyebabnya.
4. **Ubah.** Lakukan perubahan kecil yang diminta, lalu amati hasilnya.

Nilai LK **tidak** bergantung pada benar atau salahnya tebakan. Yang dinilai adalah kejujuran tebakan dan kualitas penjelasan selisihnya. Tebakan yang salah tetapi dijelaskan dengan benar bernilai penuh. Output yang "ditebak" setelah program dijalankan biasanya mudah dikenali karena penjelasannya kosong.

L2.a – `FlowDemo`, bagian A, B, C. File `src/shop/demo/FlowDemo.java` (sudah tersedia, jangan diubah):

```
// Part A: try-catch-finally
static void partA(String input) {
    try {
        System.out.println("A1 mulai");
        int quantity = Integer.parseInt(input);
        System.out.println("A2 jumlah = " + quantity);
    } catch (NumberFormatException e) {
        System.out.println("A3 bukan angka");
    } finally {
```

```
        System.out.println("A4 finally");
    }
    System.out.println("A5 lanjut");
}
```

```
// Part B: a catch that does not match
static long partB(long total, String monthsText) {
    try {
        int months = Integer.parseInt(monthsText);
        System.out.println("B1 bulan = " + months);
        long monthly = total / months;
        System.out.println("B2 cicilan = " + monthly);
        return monthly;
    } catch (NumberFormatException e) {
        System.out.println("B3 bukan angka");
        return 0;
    } finally {
        System.out.println("B4 finally");
    }
}
```

```
// Part C: return inside try
static int partC(int stock) {
    try {
        System.out.println("C1 cek stok");
        return stock - 1;
    } finally {
        System.out.println("C2 finally");
    }
}
```

dan `main`-nya:

```
System.out.println("== Bagian A ==");
partA("12");
partA("12rb");

System.out.println("== Bagian B ==");
try {
    System.out.println("B5 hasil = " + partB(300_000, "3"));
    System.out.println("B5 hasil = " + partB(300_000, "0"));
    System.out.println("B6 baris ini?");
} catch (ArithmeticException e) {
    System.out.println("B7 main menangkap: " + e.getMessage());
}

System.out.println("== Bagian C ==");
System.out.println("C3 sisa = " + partC(5));
```

Tulis prediksi bagian A, B, dan C di LK. Setelah itu baru jalankan `FlowDemo` (tombol **Run**).

Petunjuk untuk bagian B: perhatikan baris `"B5 hasil = " + partB(...)`. Method `partB` harus selesai dulu sebelum `println` di `main` bisa mencetak apa pun.

L2.b – `CariProdukDemo`, bagian D (beberapa catch, argumen berbeda).

```
String[] skus = {"KOPI-001", "RDG-002", "TEH-004"};
try {
    if (args.length == 0) {
        throw new IllegalArgumentException("Berikan satu nomor produk.");
```

```
    }
    int number = Integer.parseInt(args[0]);
    System.out.println("D1 SKU: " + skus[number - 1]);
} catch (NumberFormatException e) {
    System.out.println("D2 nomor harus bilangan bulat");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("D3 nomor di luar katalog: " + e.getMessage());
} catch (IllegalArgumentException e) {
    System.out.println("D4 " + e.getMessage());
} finally {
    System.out.println("D5 pencarian selesai");
}
```

Prediksi output untuk **empat** perintah berikut, lalu jalankan dari terminal:

```
java -cp bin shop.demo.CariProdukDemo 2
java -cp bin shop.demo.CariProdukDemo abc
java -cp bin shop.demo.CariProdukDemo 8
java -cp bin shop.demo.CariProdukDemo
```

Ingat: `NumberFormatException` adalah **turunan** `IllegalArgumentException`. Untuk input `abc`, catch mana yang jalan, D2 atau D4? Kenapa?

L2.c – Ubah. Pindahkan blok `catch (IllegalArgumentException e)` ke posisi **paling atas** (sebelum catch `NumberFormatException`). Simpan, lalu kompilasi (lihat panel **Problems** atau jalankan lagi). Catat pesan error compiler-nya di LK, lalu **kembalikan** urutannya seperti semula.

Kesalahan umum L2 (baca **setelah** menjalankan program)

Anggapan keliru	Yang benar
"Setelah catch selesai, Java kembali ke baris sesudah baris yang gagal."	Tidak pernah kembali. Sisa blok `try` dilewati.
"`finally` hanya jalan kalau ada exception."	`finally` jalan saat berhasil, saat ditangkap, saat tidak tertangkap, bahkan saat ada `return`.
"Kalau catch-nya tidak cocok, program langsung mati."	Exception naik ke pemanggil. Program baru mati kalau sampai `main` pun tidak ada yang menangkap.
"Semua catch yang cocok dijalankan."	Hanya catch **pertama** yang cocok, dibaca dari atas.

> ***Isi LK – bagian L2:*** *untuk A, B, C, dan D (4 perintah): prediksi, output sebenarnya, cocok atau tidak, dan penjelasan selisih. Ditambah pesan error compiler dari L2.c beserta artinya dalam satu kalimat.*

L3 – Custom exception: toko yang bisa menjelaskan kegagalannya (35 menit)

Tujuan: mengganti `boolean` pada pembayaran dan checkout dengan exception yang menyebut penyebab kegagalan, serta memastikan stok kembali utuh saat checkout gagal.

Class yang `extends Exception` (bukan `RuntimeException`) adalah **checked exception**: method yang melemparnya wajib menulis `throws`, dan pemanggilnya wajib menangkap atau ikut menulis `throws`. Toko memakai checked exception untuk kegagalan yang masih bisa ditangani pembeli (stok habis, saldo kurang). Kesalahan pemakaian tetap memakai unchecked exception bawaan Java.

Analogi. `false` itu seperti dokter yang hanya berkata "pasien sakit". `InsufficientBalanceException` adalah diagnosisnya: "saldo kurang Rp15.000". Dengan diagnosis, kasir tahu tindakan yang tepat, misalnya menawarkan metode bayar lain.

L3.a – Keluarga exception toko

Buat folder `src/shop/exception`. Lalu buat empat class berikut (bandingkan dengan diagram di bagian B).

`ShopException` (contoh lengkap):

```
package shop.exception;

/** Base type for every business failure of the shop (checked). */
public class ShopException extends Exception {
    private static final long serialVersionUID = 1L;

    public ShopException(String message) {
        super(message);
    }

    public ShopException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

`serialVersionUID` cukup ditulis agar compiler tidak memberi peringatan; penjelasannya di luar cakupan kuliah ini.

`OutOfStockException` (contoh lengkap). Exception boleh membawa data, sehingga pemanggil tidak perlu membaca-baca teks pesan:

```
package shop.exception;

public class OutOfStockException extends ShopException {
    private static final long serialVersionUID = 1L;

    private final String sku;
    private final int requested;
    private final int available;

    public OutOfStockException(String sku, int requested, int available) {
        if (requested <= available) { // Java 25: a check before super(...) (JEP 513)
            throw new IllegalArgumentException("Stock is sufficient, this is not an
error");
        }
        super("Stok " + sku + " tidak cukup: diminta " + requested + ", tersedia " +
available);
        this.sku = sku;
        this.requested = requested;
        this.available = available;
    }

    public String getSku() { return sku; }
    public int getRequested() { return requested; }
    public int getAvailable() { return available; }
}
```

> ***Catatan Java modern – flexible constructor bodies.*** *Sebelum Java 25, `super(...)` wajib menjadi baris pertama konstruktor. Sejak Java 25 (JEP 513), pemeriksaan argumen boleh ditulis* ***sebelum***

> *`super(...)`, asalkan belum memakai `this`. Di sini, membuat `OutOfStockException` padahal stok cukup adalah kesalahan programmer, jadi ditolak dengan exception unchecked. Anda tidak wajib meniru gaya ini; kode tanpa `if` tersebut juga benar. Sumber: Oracle, JEP 513: Flexible Constructor Bodies.*

`PaymentFailedException` (TULIS SENDIRI). `extends ShopException`, punya `serialVersionUID`, dan dua konstruktor yang sama bentuknya dengan `ShopException`: `(String message)` dan `(String message, Throwable cause)`.

`InsufficientBalanceException` (TULIS SENDIRI).

- `extends PaymentFailedException` (saldo kurang adalah **jenis khusus** dari bayar gagal), punya `serialVersionUID`.
- Konstruktor `InsufficientBalanceException(long required, long available)` dengan pesan persis: `Saldo tidak cukup: perlu Rp<required>, tersedia Rp<available>`.
- Getter `getRequired()`, `getAvailable()`, dan `getShortfall()` (= `required - available`).

L3.b – `Inventory`: pesan dan kembalikan stok (TULIS SENDIRI)

File `src/shop/service/Inventory.java` sudah punya `addStock` dan `available`. Ganti isi dua method TODO:

- `public void reserve(String sku, int qty) throws OutOfStockException`
  - `qty <= 0` → `IllegalArgumentException("Quantity must be positive")` (kesalahan programmer, unchecked).
  - `qty` melebihi stok → `OutOfStockException(sku, qty, stokSaatIni)` (kegagalan bisnis, checked).
  - Selain itu, kurangi stok.
- `public void release(String sku, int qty)`: tolak `qty <= 0` dengan `IllegalArgumentException`, lalu tambahkan `qty` ke stok.

Prinsipnya: **periksa dulu, ubah data terakhir**. Kalau stok dikurangi sebelum diperiksa, kegagalan meninggalkan data yang sudah terlanjur rusak.

L3.c – `pay` berhenti menjawab `boolean`

Ubah `src/shop/payment/PaymentMethod.java` menjadi:

```
package shop.payment;

import shop.exception.PaymentFailedException;

/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();

    /** Completes normally on success; throws when the payment fails. */
    void pay(long amount) throws PaymentFailedException;
}
```

Simpan, lalu lihat panel **Problems**. Akan muncul **sekitar 10 error di 4 file**: `EWalletPayment`, `BankTransferPayment`, `CreditCardPayment`, dan `CheckoutService`. Jangan panik, satu perubahan kontrak (*interface*) memang memaksa semua pihak yang memakainya ikut berubah. Catat jumlah error dan nama file-nya di LK sebelum memperbaiki. Perbaiki dari file pertama di daftar.

`BankTransferPayment.pay` (contoh):

```
@Override
public void pay(long amount) throws PaymentFailedException {
    if (amount <= 0) {
        throw new IllegalArgumentException("Amount must be positive");
    }
    if (!online) {
        throw new PaymentFailedException(bankName + " sedang gangguan, coba lagi
nanti");
    }
}
```

`EWalletPayment.pay` (TULIS SENDIRI): `public void pay(long amount) throws InsufficientBalanceException`. Jika `amount <= 0`, lempar `IllegalArgumentException`. Jika `amount` melebihi saldo, lempar `InsufficientBalanceException(amount, balance)` **tanpa** mengubah saldo. Selain itu, kurangi saldo.

`CreditCardPayment.pay` (TULIS SENDIRI): sama seperti e-wallet, tetapi yang dibandingkan adalah sisa limit `creditLimit - usedCredit`. Jika berhasil, `usedCredit` bertambah. Limit kurang juga memakai `InsufficientBalanceException`, dengan sisa limit sebagai nilai `available`. Tidak perlu membuat class exception baru, karena bagi pemanggil artinya sama.

Mengapa boleh menulis `throws InsufficientBalanceException` padahal interface menulis `throws PaymentFailedException`? Karena itu turunannya (lebih sempit). Yang **dilarang**: menambah checked exception baru yang tidak ada di interface, atau memperluasnya menjadi `throws Exception`. Inilah contoh nyata Liskov Substitution Principle: implementasi tidak boleh mengejutkan pemakai kontrak.

L3.d – `CheckoutService`: pesan stok, bayar, batalkan kalau gagal (contoh, pahami baik-baik)

Tambahkan field dan konstruktor berikut, lalu ganti method `checkout` yang lama:

```
private final Inventory inventory;

public CheckoutService(Inventory inventory) {
    if (inventory == null) {
        throw new IllegalArgumentException("Inventory is required");
    }
    this.inventory = inventory;
}
```

```
/** Reserves stock, then pays. Returns the amount paid. */
public long checkout(List<Product> items, ShippingService shipping, PaymentMethod
payment)
        throws OutOfStockException, PaymentFailedException {
    if (items.isEmpty()) {
        throw new IllegalStateException("Cart is empty");
    }
    List<Product> reserved = new ArrayList<>();
    try {
        for (Product product : items) {
            inventory.reserve(product.getSku(), 1);
            reserved.add(product);
        }
        long amount = total(items, shipping);
        payment.pay(amount);
        return amount;
    } catch (OutOfStockException | PaymentFailedException e) {
        for (Product product : reserved) {
            inventory.release(product.getSku(), 1); // undo the reservation
```

```
        }
        throw e; // rethrow: the caller decides what to tell the customer
    }
}
```

Tambahkan `import java.util.ArrayList;` dan import tiga exception yang dipakai. Tiga hal yang akan ditanyakan di LK dan saat pengecekan lisan:

1. Keranjang kosong memakai `IllegalStateException` (unchecked), karena itu kesalahan **pemanggil**. Pemanggil seharusnya memeriksa dulu.
2. Setiap produk dipesan satu per satu. Kalau barang ketiga habis, dua barang pertama sudah terlanjur dipesan, sehingga harus dikembalikan.
3. `throw e` melempar ulang exception **yang sama**. `CheckoutService` tidak tahu cara bicara ke pembeli; tugasnya hanya membereskan stok.

Setelah L3.d, `ShopApp` ikut error (konstruktor lama dan `boolean paid = ...`). Itu normal dan diperbaiki di L3.e.

L3.e – `ShopApp`: satu tempat untuk bicara ke pembeli

Method `buy` (TULIS SENDIRI). Tambahkan ke `ShopApp`:

```
public static String buy(CheckoutService checkout, List<Product> cart,
                         ShippingService shipping, PaymentMethod payment)
```

Method ini memanggil `checkout.checkout(...)` dan **mengembalikan** satu baris pesan (tidak mencetaknya):

Hasil	Pesan yang dikembalikan
Berhasil	`[OK] <payment.getName()> membayar Rp<jumlah dibayar>`
`InsufficientBalanceException`	`[SALDO] <getMessage()> (kurang Rp<getShortfall()>)`
`PaymentFailedException` lainnya	`[BAYAR] <getMessage()>`
`OutOfStockException`	`[STOK] <getMessage()>`

Urutan catch menentukan hasil. Jangan menangkap `Exception` atau `IllegalStateException` di sini. Kesalahan pemanggil harus tetap terlihat, bukan disamarkan menjadi pesan.

Perbarui `main` (contoh). Setelah `List<Product> catalog = sampleCatalog();`, siapkan stok dari NIM:

```
Inventory inventory = new Inventory();
for (Product product : catalog) {
    inventory.addStock(product.getSku(), StudentInfo.stockFor(product.getSku()));
}
```

Ubah baris cetak katalog menjadi `System.out.println((i + 1) + ". " + product.getInfo() + " | stok " + inventory.available(product.getSku()));` (ambil dulu `Product product = catalog.get(i);`). Lalu ganti bagian setelah loop input dengan:

```
if (cart.isEmpty()) { // check the precondition before calling checkout
    System.out.println("\nKeranjang kosong. Sampai jumpa!");
    return;
}

CheckoutService checkout = new CheckoutService(inventory);
ShippingService regular = new RegularShipping();
```

```
EWalletPayment wallet = new EWalletPayment("DompetKu", StudentInfo.walletBalance());
System.out.println("\nKeranjang: " + cart.size() + " barang, total Reguler Rp"
        + checkout.total(cart, regular));
System.out.println(buy(checkout, cart, regular, wallet));
System.out.println("Sisa saldo: Rp" + wallet.getBalance());
```

Contoh output (NIM 2401010057, membeli 1 kopi + 1 rendang). Bandingkan dengan `Pembayaran berhasil? false` di L1:

```
== Toko Online UMKM ==
1. [Fisik] Kopi Arabika Gayo 250g - Rp85000 | stok 3
2. [Fisik] Rendang Kemasan 250g - Rp120000 | stok 2
3. [Fisik] Teh Kayu Aro 100g - Rp35000 | stok 5
Nomor produk (0 = selesai): 1
Jumlah: 1
  + 1 x Kopi Arabika Gayo 250g
Nomor produk (0 = selesai): 2
Jumlah: 1
  + 1 x Rendang Kemasan 250g
Nomor produk (0 = selesai): 0

Keranjang: 2 barang, total Reguler Rp215000
[SALDO] Saldo tidak cukup: perlu Rp215000, tersedia Rp200000 (kurang Rp15000)
Sisa saldo: Rp200000
```

L3.f – Uji enam skenario berurutan

Salin `bahan/L3/ScenarioRunner.java` ke `src/shop/app/`. Program ini menjalankan enam pembelian **berturut-turut** pada stok dan e-wallet yang **sama**. Akibatnya, hasil satu skenario memengaruhi skenario berikutnya.

No.	Keranjang	Pengiriman	Pembayaran
S1	Kopi + Rendang	Reguler	e-wallet (saldo dari NIM)
S2	Kopi x3	Reguler	transfer bank (sedang gangguan)
S3	Rendang x2	Reguler	e-wallet
S4	Kopi	Ekspres (Rp18000/kg)	kartu kredit, limit Rp100000
S5	kosong	Reguler	e-wallet
S6	Teh x2	Reguler	e-wallet

Harga: kopi Rp85000 (300 g), rendang Rp120000 (400 g), teh Rp35000 (150 g). Ongkir Reguler Rp10000 per kg yang dimulai, misalnya 700 g dihitung 1 kg. Stok awal teh 5; stok awal kopi, rendang, dan saldo dari `NimParams`.

Isi dulu kolom prediksi di LK untuk setiap skenario (jenis hasil `[OK]`/`[SALDO]`/`[BAYAR]`/`[STOK]`/`[INVALID]`, serta stok dan saldo setelahnya), **baru** jalankan `ScenarioRunner`. S5 dicetak sebagai `[INVALID]` karena `buy` memang tidak menangkap `IllegalStateException`; `ScenarioRunner` yang menangkapnya agar skenario berikutnya tetap berjalan.

Kesalahan umum L3

Salah	Benar	Alasan
`ShopException extends RuntimeException`	`extends Exception`	Menjadi unchecked; compiler berhenti mengingatkan

Salah	Benar	Alasan
Konstruktor exception tanpa `super(message)`	Teruskan pesan ke `super(...)`	`getMessage()` menjadi `null`
`catch (PaymentFailedException e)` di atas `catch (InsufficientBalanceException e)`	Yang spesifik di atas	Compile error: *exception InsufficientBalanceException has already been caught*
`public void pay(long amount) throws Exception`	`throws InsufficientBalanceException`	Compile error: implementasi tidak boleh memperluas `throws` milik interface
Di `checkout`: catch lalu `System.out.println(...)` atau `return 0`	Kembalikan stok, lalu `throw e`	Pemanggil tidak pernah tahu checkout gagal
Lupa mengembalikan stok di catch	`release` semua yang sudah dipesan	Stok "hilang" setiap kali pembayaran gagal (SelfCheck L3.11)
`buy` menangkap `Exception`	Tangkap tiga tipe spesifik	Ikut menyembunyikan bug, misalnya keranjang kosong

Checkpoint L3: SelfCheck L3.1–L3.13 lulus, total **18 / 24**:

```
-- L3 - Custom exception, Inventory, checkout
  [LULUS] L3.1 ShopException adalah checked exception dengan 2 konstruktor
  [LULUS] L3.2 Hierarki exception toko + serialVersionUID
  [LULUS] L3.3 Exception membawa data (sku, requested, available, shortfall)
  [LULUS] L3.4 Inventory.reserve: stok kurang -> OutOfStockException, stok tetap
  [LULUS] L3.5 Inventory.release mengembalikan stok
  [LULUS] L3.6 PaymentMethod.pay(long) bertipe void dan throws PaymentFailedException
  [LULUS] L3.7 EWalletPayment: saldo kurang -> InsufficientBalanceException
  [LULUS] L3.8 CreditCardPayment: limit kurang -> InsufficientBalanceException
  [LULUS] L3.9 CheckoutService(Inventory); checkout(...) mengembalikan long + throws
  [LULUS] L3.10 Checkout berhasil: jumlah dibayar benar, stok berkurang
  [LULUS] L3.11 Checkout gagal: stok dikembalikan, exception asli dilempar ulang
  [LULUS] L3.12 Checkout keranjang kosong -> IllegalStateException
  [LULUS] L3.13 ShopApp.buy(...): urutan catch dan pesan [OK]/[SALDO]/[BAYAR]/[STOK]
...
=== HASIL: 18 / 24 pemeriksaan lulus (nilai SelfCheck 38 / 50) ===
```

Di checkpoint ini, asisten atau dosen dapat meminta mahasiswa menjelaskan method `checkout` secara lisan selama 1–2 menit.

> ***Isi LK – bagian L3:*** *(a) jumlah error dan nama file setelah `PaymentMethod` diubah, serta alasannya dalam satu kalimat; (b) tabel prediksi dan hasil enam skenario `ScenarioRunner`; (c) mengapa `checkout` mengembalikan stok lalu `throw e`, bukan mencetak pesan; (d) mengapa catch `InsufficientBalanceException` harus di atas catch `PaymentFailedException`.*

L4 – try-with-resources: katalog dari file CSV (20 menit)

Tujuan: membaca katalog dari file dengan try-with-resources, melewati baris yang rusak tanpa menghentikan toko, dan membungkus kegagalan membaca file menjadi exception toko yang tetap menyimpan penyebab aslinya.

Resource (file, koneksi) adalah sesuatu yang dipinjam dan wajib dikembalikan. Resource yang ditulis di dalam kurung `try (...)` ditutup otomatis, baik blok berhasil maupun gagal. Konstruktor `(String message, Throwable cause)` membuat exception baru yang **menyimpan** exception lama sebagai penyebab (*chained exception*).

Analogi: Karyawan meminjam kunci loker saat mulai kerja. Kalau ia pulang mendadak karena ada masalah dan lupa mengembalikan kuncinya, loker terkunci untuk orang berikutnya. try-with-resources adalah satpam yang selalu mengambil kunci itu di pintu keluar.

File `katalog.csv` di folder proyek (sejajar dengan `src`) berisi 7 baris dengan format `sku;nama;harga;beratGram`. Tiga di antaranya sengaja rusak dengan cara yang berbeda:

```
KOPI-001;Kopi Arabika Gayo 250g;85000;300
RDG-002;Rendang Kemasan 250g;120000;400
KUE-003;Kue Lapis Legit;95rb;800
TEH-004;Teh Kayu Aro 100g;35000;150
BATIK-005;Kain Batik Tulis
SAMB-006;Sambal Lado Mudo 200g;-45000;250
KRIP-007;Keripik Balado 200g;30000;200
```

L4.a – `CatalogLoadException` (TULIS SENDIRI). Di `shop.exception`: `extends ShopException`, punya `serialVersionUID`, dan **satu** konstruktor `(String message, Throwable cause)`. Hanya ada satu konstruktor karena exception ini selalu membungkus penyebab lain.

L4.b – Lengkapi `CatalogReader`. Salin `bahan/L4/CatalogReader.java` ke `src/shop/io/` (buat folder `io`). Method `parse` dan `getSkippedCount` sudah jadi. Lengkapi `load` mengikuti TODO di dalamnya:

```
public List<Product> load(Path file) throws CatalogLoadException {
    List<Product> products = new ArrayList<>();
    skippedCount = 0;
    // TODO L4.b
    // 1. Open the file with try-with-resources: BufferedReader reader =
Files.newBufferedReader(file)
    // 2. Read line by line (reader.readLine() returns null at the end) and count the
line number.
    // 3. Parse each line inside its OWN try-catch (IllegalArgumentException):
    //    a broken line -> skippedCount++, LOG.warning("Baris <n> dilewati:
<message>"), continue.
    // 4. IOException -> throw new CatalogLoadException("Katalog gagal dibaca: " +
file, e)
    return products;
}
```

Dua `try` dengan tugas berbeda:

Masalah	Contoh	Cara menangani	Alasan
Satu baris rusak	`95rb`, kolom kurang, harga negatif	Catat ke log, lewati, lanjut ke baris berikutnya	Satu produk rusak tidak boleh membuat toko tutup
File tidak bisa dibaca	File tidak ada	Bungkus menjadi `CatalogLoadException`, lempar ke atas	Tanpa katalog, toko memang tidak bisa buka

Satu `catch (IllegalArgumentException e)` cukup untuk ketiga baris rusak: `NumberFormatException` adalah turunannya, dan validasi konstruktor `Product` (harga negatif) juga melempar `IllegalArgumentException`. Peringatan kuning "never used" di VS Code hilang setelah TODO selesai.

L4.c – ShopApp membaca katalog dari file (contoh). Ganti baris `List<Product> catalog = sampleCatalog();` dengan:

```
// L4: the catalog comes from a file; a different file can be given as an argument
Path catalogFile = Path.of(args.length > 0 ? args[0] : "katalog.csv");
CatalogReader reader = new CatalogReader();
List<Product> catalog;
try {
    catalog = reader.load(catalogFile);
} catch (CatalogLoadException e) {
    System.out.println("Toko tidak bisa dibuka: " + e.getMessage());
    System.out.println("Penyebab asli: " + e.getCause());
    return;
}
```

Tambahkan satu baris tepat di bawah judul `== Toko Online UMKM ==`:

```
System.out.println("(" + catalog.size() + " produk, " + reader.getSkippedCount()
        + " baris katalog dilewati)");
```

Lalu hapus method `sampleCatalog()` beserta `import shop.model.PhysicalProduct;`, dan tambahkan import `java.nio.file.Path`, `shop.exception.CatalogLoadException`, serta `shop.io.CatalogReader`.

Contoh output (baris tanggal di log akan berbeda; teks setelah titik dua adalah ketikan pembeli):

```
Nov 08, 2026 11:08:41 AM shop.io.CatalogReader load
WARNING: Baris 3 dilewati: For input string: "95rb"
Nov 08, 2026 11:08:41 AM shop.io.CatalogReader load
WARNING: Baris 5 dilewati: expected 4 fields, found 2
Nov 08, 2026 11:08:41 AM shop.io.CatalogReader load
WARNING: Baris 6 dilewati: Price must not be negative
== Toko Online UMKM ==
(4 produk, 3 baris katalog dilewati)
1. [Fisik] Kopi Arabika Gayo 250g - Rp85000 | stok 3
2. [Fisik] Rendang Kemasan 250g - Rp120000 | stok 2
3. [Fisik] Teh Kayu Aro 100g - Rp35000 | stok 5
4. [Fisik] Keripik Balado 200g - Rp30000 | stok 5
Nomor produk (0 = selesai): 1
Jumlah: 1
  + 1 x Kopi Arabika Gayo 250g
Nomor produk (0 = selesai): 4
Jumlah: 2
  + 2 x Keripik Balado 200g
Nomor produk (0 = selesai): 0

Keranjang: 3 barang, total Reguler Rp155000
[OK] E-Wallet DompetKu membayar Rp155000
Sisa saldo: Rp45000
```

Pesan `WARNING` dicetak oleh `Logger` ke saluran error, sehingga di terminal VS Code bisa tampil berwarna dan urutannya kadang bercampur dengan output biasa. Itu normal.

L4.d – Ketika file tidak ada. Jalankan dari terminal dengan nama file yang tidak ada:

```
java -cp bin shop.app.ShopApp katalog-cadangan.csv
```

Tempel outputnya di LK, lalu jawab: apa isi `e.getMessage()` dan apa isi `e.getCause()`? Kalau `ShopApp` hanya mencetak `e.getCause().getMessage()`, apa yang akan dilihat pengguna, dan mengapa itu kurang membantu? Petunjuk: `getMessage()` dari exception "file tidak ditemukan" di Java hanya berisi **nama file**.

Kalau waktu menipis, mahasiswa boleh lanjut ke L5 setelah L4.c dan kembali ke L4.d bila masih sempat.

Kesalahan umum L4

Salah	Benar	Alasan
`try-catch` per baris diletakkan **di luar** `while`	Di **dalam** `while`, mengelilingi `parse(line)`	Satu baris rusak menghentikan pembacaan seluruh file
`throw new CatalogLoadException("...", null)`	`new CatalogLoadException("...", e)`	Penyebab asli hilang; tidak ada yang tahu kenapa katalog gagal
`catch (Exception e)` di dalam loop	`catch (IllegalArgumentException e)`	Ikut menelan bug, misalnya `NullPointerException`
`reader.close()` ditulis di akhir blok `try` biasa	try-with-resources	Tidak dijalankan kalau exception terjadi lebih dulu
`katalog.csv` dipindah ke dalam `src`	Biarkan di folder proyek	File dicari dari folder kerja, yaitu folder proyek

Checkpoint L4: SelfCheck L4.1–L4.4 lulus, total **22 / 24**:

```
-- L4 - try-with-resources dan katalog CSV
  [LULUS] L4.1 CatalogLoadException turunan ShopException dan menyimpan cause
  [LULUS] L4.2 katalog.csv: 4 produk terbaca, 3 baris rusak dilewati
  [LULUS] L4.3 File tidak ada -> CatalogLoadException dengan cause IOException
  [LULUS] L4.4 CatalogReader membuka file dengan try-with-resources
...
=== HASIL: 22 / 24 pemeriksaan lulus (nilai SelfCheck 46 / 50) ===
```

> ***Isi LK – bagian L4:*** *(a) tiga baris katalog yang dilewati beserta jenis masalahnya dan exception yang tertangkap; (b) output L4.d dan jawaban tentang `getMessage()` vs `getCause()`.*

L5 – Evaluasi kode: `FragileShop` (15 menit)

Tujuan: mengenali kode penanganan exception yang "berjalan" tetapi menyembunyikan masalah, menjelaskan dampaknya, lalu memperbaikinya.

Tiga aturan emas: (1) tangkap exception di tempat yang **bisa mengambil keputusan**; (2) **jangan pernah** menelan exception diam-diam; (3) ubah exception ke "bahasa" lapisan yang menerimanya, tanpa membuang penyebab aslinya.

L5.a – Jalankan. Salin `bahan/L5/FragileShop.java` ke `src/review/` (buat folder `review`), lalu jalankan:

```
1) Bayar Rp85000 selesai. Sisa saldo: Rp50000
2) Kopi tersedia? true
3) Gagal: For input string: "dua"
   Gagal: Stok KOPI-001 tidak cukup: diminta 5, tersedia 1
4) Katalog gagal dibaca: katalog-lama.csv | penyebab: null
```

Sekilas semuanya tampak wajar. Baca baris 1 sekali lagi: saldo Rp50000 dipakai membayar Rp85000, "selesai", dan saldonya tetap Rp50000. Apa yang sebenarnya terjadi?

L5.b – Review (10 menit). Ini empat method yang diuji `main`:

```
/** Pays an order. */
public static void payOrder(PaymentMethod payment, long amount) {
```

```
    try {
        payment.pay(amount);
    } catch (PaymentFailedException _) {
    }
}

/** Is at least one item in stock? */
public static boolean isInStock(Inventory inventory, String sku) {
    try {
        inventory.reserve(sku, 1);
        inventory.release(sku, 1);
        return true;
    } catch (OutOfStockException e) {
        return false;
    }
}
```

```
/** Reads the first line (the header) of a catalog file. */
public static String readCatalogHeader(Path file) throws CatalogLoadException {
    try (BufferedReader reader = Files.newBufferedReader(file)) {
        return reader.readLine();
    } catch (IOException e) {
        throw new CatalogLoadException("Katalog gagal dibaca: " + file, null);
    }
}

/** Adds items to the cart from what the customer typed. */
public static String addToCart(Inventory inventory, String sku, String quantityText)
{
    try {
        int quantity = Integer.parseInt(quantityText);
        inventory.reserve(sku, quantity);
        return "Ditambahkan " + quantity + " x " + sku;
    } catch (OutOfStockException | IllegalArgumentException e) {
        return "Gagal: " + e.getMessage();
    }
}
```

Setiap method mengandung **satu** masalah utama. Untuk masing-masing, isi tabel temuan di LK: nama method, apa masalahnya, apa dampaknya (bagi pembeli atau bagi programmer yang memperbaiki bug), dan bagaimana perbaikannya. Pertanyaan pemandu:

- Bisakah sebuah kegagalan hilang tanpa jejak?
- Apakah exception dipakai untuk menjawab pertanyaan ya/tidak yang sebenarnya bisa ditanyakan langsung? Apakah ada efek sampingnya?
- Apakah penyebab asli ikut dibawa saat exception dibungkus?
- Apakah kesalahan ketik pembeli dan stok habis ditangani dengan cara dan pesan yang sama? Apakah pesan teknis sampai ke pembeli?

L5.c – Perbaiki dua method (5 menit).

1. `payOrder`: jangan menelan kegagalan. Hapus try-catch dan tambahkan `throws PaymentFailedException`. Compiler kemudian menolak `main` (*unreported exception PaymentFailedException; must be caught or declared to be thrown*). Itulah checked exception sedang bekerja. Di `main`, tangkap exception itu dan cetak `1) Pembayaran gagal: <pesan>`.
2. `readCatalogHeader`: sertakan penyebab aslinya.

Perbaikan `isInStock` dan `addToCart` cukup ditulis di LK. Kalau masih ada waktu, kerjakan juga di kode.

Kesalahan umum L5

Salah	Benar	Alasan
Catch kosong diganti `e.printStackTrace()`	Lempar ke pemanggil, atau tangani sungguh-sungguh	Program tetap lanjut seolah pembayaran berhasil
Catch kosong diganti `throw new RuntimeException(e)`	`throws PaymentFailedException`	Kegagalan bisnis yang bisa ditangani berubah menjadi crash
`_` dianggap membuat catch kosong menjadi aman	`_` hanya menyatakan objeknya tidak dipakai	Kegagalan tetap ditelan

Checkpoint L5: SelfCheck lulus semua, **24 / 24**:

```
-- L5 - Evaluasi kode FragileShop
  [LULUS] L5.1 FragileShop.payOrder tidak lagi menelan kegagalan bayar
  [LULUS] L5.2 FragileShop.readCatalogHeader menyertakan penyebab asli

=== HASIL: 24 / 24 pemeriksaan lulus (nilai SelfCheck 50 / 50) ===
```

> ***Isi LK – bagian L5:*** *tabel empat temuan (`payOrder`, `isInStock`, `readCatalogHeader`, `addToCart`) dan output `FragileShop` setelah diperbaiki.*

Penutup (5 menit)

1. Jalankan SelfCheck terakhir kali. Salin baris `=== HASIL ...` dan baris `=== TANTANGAN ...` ke LK.
2. Jawab tiga pertanyaan refleksi singkat di akhir LK.
3. Kumpulkan sesuai bagian E di bawah, **walaupun belum semua pemeriksaan lulus**. Pekerjaan yang belum selesai tetap dinilai sesuai bagian yang sudah benar.

D. Pengumpulan dan Penilaian

Kumpulkan sebelum sesi ditutup:

1. Zip folder `src` dengan nama `P07_<NIM>_<Nama>.zip`.
2. Simpan Lembar Kerja Praktikum sebagai PDF dengan nama `LK-P07-<NIM>.pdf`.
3. Unggah keduanya ke Google Drive masing-masing, share dan kirimkan tautan linknya ke Pengumpulan LKP 07 di **CODATA LAB**.

Komponen	Bobot	Cara menilai
SelfCheck	50	(butir lulus ÷ 24) × 50, dibulatkan; angka ini sudah dicetak SelfCheck
Lembar Kerja	50	L1 8, L2 12, L3 10, L4 6, L5 10, refleksi 4. Bagian L0 (identitas) wajib diisi, tanpa poin
Nilai akhir	**maks. 100**	SelfCheck + LK

Kebijakan keaslian. Output `ScenarioRunner`, `ShopApp`, dan `NimParams` bergantung pada NIM, dan dosen memegang hasil yang benar untuk setiap NIM. Output yang tidak cocok dengan NIM Anda akan diperiksa ulang. Asisten atau dosen juga dapat meminta penjelasan lisan singkat di checkpoint L3.

E. Glosarium & Referensi

Glosarium

Istilah	Arti
Exception	Objek laporan masalah yang dilempar saat program berjalan
Checked exception	Turunan `Exception` (bukan `RuntimeException`); compiler mewajibkan ditangkap atau ditulis di `throws`
Unchecked exception	Turunan `RuntimeException`; tidak dipaksa compiler, biasanya menandakan kesalahan pemakaian
Stack trace	Jejak urutan pemanggilan method saat exception terjadi
Propagation (perambatan)	Exception yang tidak ditangkap naik ke method pemanggil
Custom exception	Class exception buatan sendiri untuk aturan bisnis
Cause / chained exception	Exception asli yang disimpan di dalam exception baru (`getCause()`)
Rethrow	Melempar ulang exception yang sama setelah membereskan sesuatu (`throw e`)
Precondition	Syarat yang harus dipenuhi pemanggil sebelum memanggil method, misalnya keranjang tidak kosong
Resource leak	Resource (file, koneksi) tidak dikembalikan
try-with-resources	`try (...)` yang menutup resource secara otomatis
Suppressed exception	Exception dari `close()` yang ditempelkan pada exception utama, bukan menggantikannya
Failure atomicity	Kalau operasi gagal, data kembali ke keadaan sebelum operasi
Menelan exception (*swallowing*)	Menangkap exception tanpa tindakan apa pun

Referensi

- Deitel, P. & Deitel, H. *Java How to Program, Early Objects*, ed. 11
- Bloch, J. *Effective Java*, ed. 3, Addison-Wesley
- Oracle Java Tutorials: Exceptions.
- Slide Perkuliahan “Exception Handling”